THE DEVELOPER'S THINKING PROCESS

How to Start a Backend Project

What to think, which file to make first,
and how experienced developers approach building from scratch

Banking Transaction System • Fresher Edition

1. The Core Mental Model

> *The single most important idea in this entire document: always build from the bottom up, not the top down. Your database is the foundation. Everything else is just a wrapper around it.*

What Most Beginners Do (Wrong)

- Open VS Code and create server.js immediately
- Start writing routes without knowing what data they need
- Add a database halfway through and start rewriting everything
- End up with a broken app and no idea where the bug is

What Experienced Developers Do (Right)

- Spend Day 0 thinking on paper, not coding
- Set up the database first, confirm it works
- Build utilities that have no dependencies
- Build one complete feature end-to-end before touching the next
- Test at every layer before moving up

The Question Chain — Think This Before Writing Any Code

Every experienced developer answers these five questions in order before opening VS Code. Write your answers on paper or in Notion.

#	Question	For This Banking Project

Q1	What are my entities?	User, Account, Transaction, FraudFlag — these become your DB tables
Q2	How are they related?	One user has one account. One account has many transactions. These become foreign keys.
Q3	What actions can happen?	Register, Login, Deposit, Withdraw, Transfer, Flag Fraud — these become API endpoints
Q4	Who is allowed to do what?	Regular users manage their own account. Admin views all fraud flags. This becomes your middleware.
Q5	What can go wrong?	Insufficient balance, duplicate email, invalid token — these become your error handling.

Once you can answer all five questions without hesitation, you are ready to open VS Code. Not before.

2. The Layer Order — Always Bottom Up

This is how every file in your project depends on another. The rule is simple: never build a layer until the layer below it is working.

ROUTES What the outside world talks to. Receives HTTP requests.

CONTROLLERS Unpacks the request. Calls the service. Sends the response.

SERVICES All business logic. The brain of your application.

MODELS All database queries. Knows SQL. Nothing else.

UTILS / DB Database connection, JWT helpers, password helpers.

▲ BUILD DIRECTION: Start here and work upward ▲

Think of it like constructing a building. You pour the foundation (database) first. Then ground floor (utils). Then first floor (models). You never start the roof before the walls are up.

3. The Five Steps Before Writing Logic

1 Think on Paper — Before Any Code (Day 0)

Spend 30 minutes answering the five questions from Section 1. Draw your tables and their relationships. List your API endpoints. This is not wasted time — this is the most important 30 minutes of the project.

Deliverable at the end of Step 1: A handwritten or Notion page with your tables, relationships, and endpoint list. Nothing coded yet.

2 Create the Database Schema — First File You Write

Your schema.sql is the very first file you create. Not server.js. Not app.js. schema.sql.

Why? Because every single file you write after this will reference column names from these tables. If you write your model before your table exists, you are just writing guesses.

```
-- This is the FIRST file you create in the project
-- Run it with: psql -U postgres -f schema.sql

CREATE TABLE users (
  id         SERIAL PRIMARY KEY,
  name       VARCHAR(100) NOT NULL,
  email      VARCHAR(100) UNIQUE NOT NULL,
  password   VARCHAR(255) NOT NULL,
  role       VARCHAR(10) DEFAULT 'user',
  created_at TIMESTAMP DEFAULT NOW()
);

-- After running this, verify it exists:
-- \dt        (lists all tables in psql)
-- \d users   (shows columns of users table)
```

Deliverable at the end of Step 2: All 4 tables exist in your database. You verified them with \dt in psql.

3 Build Your Utils — Files With Zero Dependencies

Utils are helper files that do not depend on anything else in your project. Build them first because everything above them will need them.

File	What it does and how to test it
utils/db.js	Database connection pool. Takes a query string and parameters, returns results. Test it with SELECT 1. If this works, your DB is connected.

utils/jwt.js	Two functions only: generateToken(payload) and verifyToken(token). Nothing else. Test by calling generateToken and logging the result.
utils/password.js	Two functions only: hashPassword(plainText) and comparePassword(plain, hash). Test by hashing 'hello123' and comparing it.

```
// utils/db.js — test this FIRST before writing anything else
const { Pool } = require('pg');
require('dotenv').config();

const pool = new Pool({ connectionString: process.env.DATABASE_URL });

const query = (text, params) => pool.query(text, params);

// Quick test — add this temporarily, then remove:
// query('SELECT 1').then(r => console.log('DB connected:', r.rows));

module.exports = { query };
```

Deliverable at the end of Step 3: All three util files exist and are tested individually. You have confirmed DB connection.

4 Build One Feature End-to-End Before Starting the Next

This is the biggest mistake freshers make. They try to build all models first, then all services, then all controllers. Then nothing works and they have no idea where the bug is.

The right approach: pick one feature — Registration — and build the complete vertical slice from model to route. Test it. Only then move to the next feature.

WRONG WAY	RIGHT WAY	RESULT
All models → all services → all controllers	*One full feature → test → next feature*	*Wrong = nothing works. Right = always in a working state.*

Rule: At the end of every coding session, your app must be in a working state. If you cannot test what you just built in Postman, you have gone too far too fast.

5 Three Questions Before You Code Each File

Every time you sit down to write a new file, answer these three questions first. This takes 60 seconds and saves hours of confusion.

Question	What to do with the answer
What does this file NEED?	Write your require() imports at the top. If you are importing something that does not exist yet — stop. Go build that dependency first.
What does this file GIVE?	Write your module.exports at the bottom before writing the body. Knowing what you are exporting clarifies what you need to build.
How will I test this alone?	Before wiring this file to the layer above, write a quick standalone test — even a console.log. Confirm it works in isolation first.

4. Exact File Creation Order

This is the precise order to create every file in the banking project, and the reason why each one comes when it does.

Foundation Layer — No Dependencies

1	`schema.sql`	Tables must exist before any code references column names. This is the true first file.
2	`utils/db.js`	Every model needs DB access. Build this before any model.
3	`utils/password.js`	Auth service needs this. No dependency on anything else in the project.
4	`utils/jwt.js`	Auth service needs this. No dependency on anything else in the project.
5	`.env + app.js skeleton`	Skeleton only — just Express setup and a test route. No real routes yet.

Auth Feature — First Full Vertical Slice

6	`models/userModel.js`	createUser() and findByEmail(). Depends on db.js only.

7	`services/authService.js`	registerUser() and loginUser(). Depends on userModel + password + jwt.
8	`controllers/authController.js`	Unwraps req, calls service, sends res. Depends on authService.
9	`routes/authRoutes.js`	POST /register and POST /login. Register in app.js.

✓ TEST CHECKPOINT: Open Postman. Register a user. Login. Confirm you get a JWT back. Do not proceed until this works.

Middleware — Build After You Have a Real Token to Test With

10	`middlewares/authMiddleware.js`	Extract and verify JWT. Attach user to req. You need a real token to test this — that is why it comes after login.
11	`middlewares/roleMiddleware.js`	Check req.user.role === 'admin'. One-line check. Depends on authMiddleware.

Account Feature — Second Vertical Slice

12	`models/accountModel.js`	createAccount(), getByUserId(), getBalance(). Depends on db.js.
13	`services/accountService.js`	Check if account exists before creating. Depends on accountModel.
14	`controllers/accountController.js`	Handle create and fetch requests. Depends on accountService.
15	`routes/accountRoutes.js`	All routes behind authMiddleware. Register in app.js.

✓ TEST CHECKPOINT: Create an account for your registered user. Fetch its balance. Both must work before moving on.

Transaction Feature — Hardest Part, Build Last

16	`models/transactionModel.js`	logTransaction() and getByAccountId(). Depends on db.js.

17	`services/transactionService.js`	deposit(), withdraw(), transfer(). Uses DB transactions with BEGIN/COMMIT/ROLLBACK. The most complex service in the project.
18	`controllers/transactionController.js`	Handles all three transaction types. Depends on transactionService.
19	`routes/transactionRoutes.js`	All routes behind authMiddleware. Register in app.js.

✓ TEST CHECKPOINT: Deposit 1000. Withdraw 200. Transfer 300 to another account. Check balances. Check transaction history. All must work.

Fraud Detection — Pure Logic, No DB Dependencies

20	`fraud/slidingWindow.js`	Pure JS. Queue-based logic. No database, no imports. Test standalone.
21	`fraud/cycleDetection.js`	Pure JS. Graph + DFS. No database, no imports. Test standalone.
22	`fraud/suspiciousHeap.js`	Pure JS. Sorted array as max-heap. No database, no imports. Test standalone.
23	`Wire fraud into transactionService.js`	After each transaction, call slidingWindow. If flagged, insert into fraud_flags table.

✓ TEST CHECKPOINT: Make 6 rapid transactions with one user. Check the fraud_flags table. The user should appear there.

Frontend — Always Last

24	`frontend/index.html`	Login and Register forms. Stores JWT to localStorage on success.
25	`frontend/dashboard.html`	Balance display, deposit/withdraw/transfer forms, transaction table.
26	`frontend/admin.html`	Fraud flags table, top suspicious users. Admin role check on page load.

27	`frontend/app.js`	All fetch() calls. Always attach Authorization header from localStorage.

✓ FINAL CHECKPOINT: Full end-to-end flow works in the browser. Admin panel shows fraud flags.

5. Daily Morning Checklist

Every time you sit down to code, answer these three questions before writing a single line. This keeps you on track and prevents the most common mistakes.

✓	Does my database have all the tables and columns I will need today? If no — update schema.sql first, run it, verify with \dt.
✓	Is the layer below what I am building today fully working and tested? If no — fix that layer before going up.
✓	Can I test what I am about to build in Postman when I am done? If yes — plan your test case before you start coding it.

6. The Full Thinking Process in One Page

BEFORE TOUCHING VS CODE Answer the 5 questions. Draw tables. List endpoints. 30 minutes on paper.

FIRST FILE YOU OPEN schema.sql. Create all tables. Run it. Verify with \dt. Confirm it works.

SECOND AND THIRD FILES utils/db.js then utils/password.js then utils/jwt.js. These have zero dependencies. Test each one alone.

FIRST FEATURE Build Register end-to-end: model → service → controller → route. Test in Postman. Do not move on until it works.

SECOND FEATURE Build Login. Same pattern. By the end of this you have a real JWT and can build middleware.

MIDDLEWARE Now that you have a real token, build authMiddleware and test it actually blocks unauthorised requests.

REMAINING FEATURES Accounts, then Transactions. One at a time, fully tested before the next.

FRAUD MODULES Write pure JS logic files. Test them standalone with a few hardcoded calls. Then wire into transactionService.

FRONTEND Only after the entire backend is Postman-tested. HTML + CSS + fetch(). Nothing more.

DEPLOYMENT Only after frontend is working locally. Push to GitHub. Deploy on Render. Test live URL.

7. Common Fresher Mistakes to Avoid

Mistake	Why it is a problem and what to do instead
Starting with server.js	server.js is just an entry point. Starting here means you have no idea what your data looks like yet. Start with schema.sql.
Building all routes first	Routes are the top of the stack. Building them first means building on nothing. No model = no data = no route that actually works.
Skipping Postman testing	If you only test in the browser, you cannot see your error messages properly. Test every endpoint in Postman the moment you build it.
Mixing logic into controllers	Controllers should be thin — just unwrap the request and call the service. Business logic in a controller cannot be reused or tested cleanly.
Writing models without a schema	If your table does not exist yet and you write a model that queries it, you are just writing broken code. Schema first, always.
Building frontend before backend is done	The most common mistake. Frontend work gives a false sense of progress. Always finish and test all API endpoints first.
Not committing to GitHub daily	If your laptop crashes mid-project, you lose everything. Commit at the end of every session with a meaningful message.